visiualize_performance_model

February 12, 2026

1 Figure 12. Comparisons on performance modeling

1.1 Average Prediction Error

1.1.1 Step 1. load data

[1]:
```
import os
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np

from Suika import *
import torch

policies = ["suika", "rubick", "metis"]
# policies = ["rubick"]

def load(policies):
  data = {}
  for policy in policies:
    path = f"../results/clusterA/{policy}-clusterA.pth"
    try:
      stats = torch.load(path, weights_only=False)
      data[policy] = stats
      print(f"Loaded {path}")
    except:
      print(f"Failed to load {path}")
  return data

# load
all_stats = load(policies)
```

```
/home/fake/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Loaded ../results/clusterA/suika-clusterA.pth
Loaded ../results/clusterA/rubick-clusterA.pth
```

```
Loaded ../results/clusterA/metis-clusterA.pth
```

1.1.2 Step 2. define estimate functions

[2]:
```python
def get_model_spec(job_name: str):
  if not isinstance(job_name, str):
    job_name = job_name.job_name
  return job_name.split("-it-")[0]

def suika_est(job: Job, pc: PlanConfig):
  model_spec = get_model_spec(job)
  eC = get_estimator_C_from_spec(model_spec)
  rm = get_default_rm()
  est = Estimator(rm, eC)
  return est.estimate(pc)

from Suika.external.Rubick.pred_throughput import predict_throughput
def rubick_est(job: Job, pc: PlanConfig):
  model_spec = get_model_spec(job)
  ms = ModelStrategy.from_spec(model_spec)
  # assume pc to be symmetric, which is always the case for Rubick
  pp = len(pc.acc_mesh)
  dp = len(pc.acc_mesh[0])
  tp = len(pc.acc_mesh[0][0])
  ms.pp_dp_tp = (pp, dp, tp)
  return predict_throughput(ms, output_latency=True)

from Suika.external.Metis.suika_interface import Metis_estimate
def metis_est(job: Job, pc: PlanConfig):
  model_spec = get_model_spec(job)
  mc = ModelConfig.from_spec(model_spec)
  est = Metis_estimate(pc, mc, get_default_rm())
  return est / 1000

def get_est_error(policy: str, job: Job, pc: PlanConfig, actual_latency: float):
  if policy == "suika":
    est_latency = suika_est(job, pc)
  elif policy == "rubick":
    # Suika limits the max scale-up world size to 8x user-specified world size␣
 ↪to avoid severe underutilization.
    # for a fair comparison, we also skip such cases for Rubick; this favors␣
 ↪the baseline.
    if pc.world_size() > job.usr_plan_config.world_size() * 8:
      return 0.0
    est_latency = rubick_est(job, pc)
  elif policy == "metis":
    est_latency = metis_est(job, pc)
```

```
  else:
    raise ValueError(f"Unknown policy {policy}")

  error = abs(est_latency - actual_latency) / max(actual_latency, est_latency)
  # print(f"{job.job_name} > {signed_error*100:.2f}")
  return error


def get_avg_error(policy: str, stats):
  total_error = 0.0
  count = 0
  for job, job_stats in stats.items():
    pcs = job_stats['plan_config']
    latencies = job_stats['iter_time']
    for idx, latency in latencies.items():
      if latency <= 0:
        continue
      pc = pcs[idx]
      error = get_est_error(policy, job, pc, latency)
      total_error += error
      count += 1

  if count == 0:
    return None
  return total_error / count
```

1.1.3 Step 3. evaluate for each policy and plotting

[3]:
```
avg_err_dict = {}

for policy in policies:
  stats = all_stats.get(policy, {})
  avg_error = get_avg_error(policy, stats)
  avg_err_dict[policy] = avg_error
  if avg_error is not None:
    print(f"Policy: {policy}, Average Relative Error: {avg_error*100:.2f}%")
  else:
    print(f"Policy: {policy}, No valid data to compute error.")

policy_map = {
    "suika": "Suika",
    "rubick": "Rubick",
    "metis": "Metis",
}

mpl.rcParams.update({'font.size': 20})
```

```
styles = {
    "Suika": {"color": "#34495e", "hatch": "", "label": "Suika"},
    "Rubick": {"color": "#f1c40f", "hatch": "x", "label": "Rubick"},
    "Metis": {"color": "#3498db", "hatch": ".", "label": "Metis"},
}

models = [policy_map.get(policy, policy) for policy in policies]
error_rates = [avg_err_dict.get(policy, 0.0) for policy in policies]

x = np.arange(len(models))  # 0,1,2
width = 0.55

fig, ax = plt.subplots(figsize=(6, 4))
margin_ratio = 0.4
bars = []
for i, model in enumerate(models):
    style = styles[model]
    bar = ax.bar(
        x[i],
        error_rates[i] * 100,
        color=style["color"],
        hatch=style["hatch"],
        label=style["label"],
        edgecolor="white",
        width=width,
        align="center",
    )
    bars.append(bar)

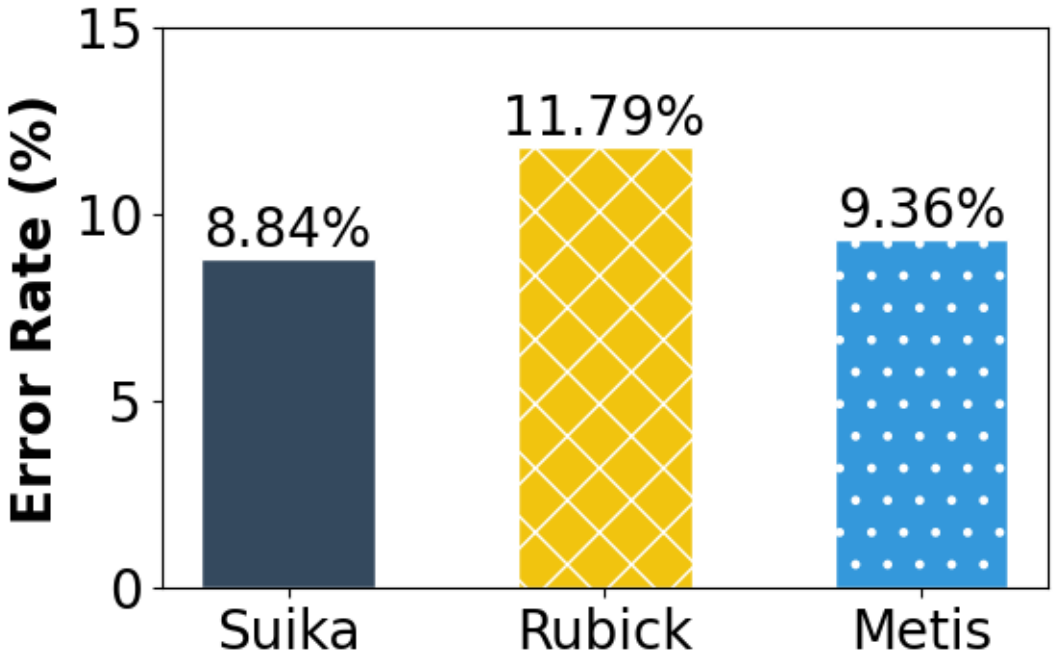
ax.set_ylim(0, 13)
ax.set_ylabel("Error Rate (%)", fontweight="bold")
ax.set_xticks(x)
ax.set_xticklabels(models)

ax.set_xlim(-margin_ratio, len(models) - 1 + margin_ratio)

for i, bar in enumerate(bars):
    height = bar[0].get_height()
    ax.text(
        bar[0].get_x() + bar[0].get_width() / 2,
        height,
        f"{error_rates[i]*100:.2f}%",
        ha="center",
        va="bottom",
        color="black"
    )
ax.set_ylim(0, 15)
```

```
plt.tight_layout()
fig_dir = "../fig"
os.makedirs(fig_dir, exist_ok=True)
plt.savefig(f"{fig_dir}/prediction_error.pdf", format="pdf")
plt.show()
```

```
Policy: suika, Average Relative Error: 8.84%
Policy: rubick, Average Relative Error: 11.79%
Policy: metis, Average Relative Error: 9.36%
```



1.2 Average Profiling Cost

Measured with external clock during profiling.

[4]:
```
time_cost_dict = {
    "Suika": 0,
    "Metis": 602,
    "Rubick": 314
}

print("\n Profiling Time Cost:")
for k, v in time_cost_dict.items():
    print(f"{k}: {v} s")
```

```
mpl.rcParams.update({'font.size': 20})

systems = list(time_cost_dict.keys())
overheads = list(time_cost_dict.values())
styles = {
    "Suika": {"color": "#34495e", "hatch": "", "label": "Suika"},
    "Rubick": {"color": "#f1c40f", "hatch": "x", "label": "Rubick"},
    "Metis": {"color": "#3498db", "hatch": ".", "label": "Metis"},
}

x = np.arange(len(systems))
width = 0.5

fig, ax = plt.subplots(figsize=(6, 4))
margin_ratio = 0.4

bars = []
for i, system in enumerate(systems):
    bar = ax.bar(
        x[i],
        overheads[i],
        width=width,
        color=styles[system]["color"],
        hatch=styles[system]["hatch"],
        edgecolor="white",
        label=styles[system]["label"],
        align="center"
    )
    bars.append(bar)

    if overheads[i] > 0:
        ax.text(
            x[i],
            overheads[i] + 10,
            f"{overheads[i]//60}min",
            ha="center",
            va="bottom",
            color="black",
        )

ax.annotate(
    "Zero Overhead",
    xy=(x[0], 0),
    xytext=(x[0] + 0.18, 100),
    arrowprops=dict(color="red", arrowstyle="->", linewidth=2),
    color="red",
    ha="center",
```

```
    fontsize=16,
    fontweight='bold',
)

ax.set_xticks(x)
ax.set_xticklabels(systems)
ax.set_ylabel("Overhead (s)", fontweight="bold")
ax.set_ylim(0, 700)
# ax.grid(True, axis="y", linestyle="--", alpha=0.7)

ax.set_xlim(-margin_ratio, len(systems) - 1 + margin_ratio)


fig_dir = "../fig"
os.makedirs(fig_dir, exist_ok=True)
filename = f"{fig_dir}/profiling_overhead.pdf"
plt.savefig(filename, dpi=600, bbox_inches="tight")
print(f"Plot saved as '{filename}'")

plt.show()
```

```
 Profiling Time Cost:
Suika: 0 s
Metis: 602 s
Rubick: 314 s
Plot saved as '../fig/profiling_overhead.pdf'
```

